2026-11-04 Next.js Proxy(예전 Middleware)와 이 앱의 접근 제어

- 날짜: 2026-11-04
- 앱: `lgcns-fe-next`
- 패키지: `next` 16.4.3, `react` 19.3.0, `next-auth` ^4.24.15 (`lgcns-fe-next/package.json`)
- 기준: 이 문서의 "현재 코드"는 지금 디스크의 파일이다. 환경 변수 값, 시크릿, 토큰 문자열은 적지 않는다.

Next.js 16은 파일 이름 `middleware.ts`를 `proxy.ts`로 바꿨다. 이 앱의 파일은 `lgcns-fe-next/proxy.ts`이고, `app/`과 같은 높이에 있다. 함수 이름은 `proxy`다. `middleware`라는 이름으로 export하면 Next는 "Proxy is missing expected function export name"으로 거절한다. 허용되는 형태는 이름 있는 `proxy` 하나, 또는 default export 하나다. 한 파일에 프록시를 여러 개 둘 수는 없다.

이 프록시는 라우트를 그리기 전에 요청을 본다. 하는 일은 두 가지다. 로그인된 사용자를 `/login`, `/signup`에서 내보내고, 로그인되지 않은 사용자를 `/cart`, `/order`에서 `/login`으로 보낸다. 그 외 경로는 통과시킨다.

1. 파일이 언제 실행되는가

이 앱 소스에는 프록시를 불러오는 `<script>`가 없다. Next가 파일 규약으로 `proxy.ts`를 서버에서 읽는다.

파일	선언	실행 시점	HTML script와의 차이
lgcns-fe-next/proxy.ts	export async function proxy(request: NextRequest) 와 export const config	매칭된 요청마다, 페이지·레이아웃을 그리기 전에 Node.js에서 실행된다. await getToken(...) 때문에 함수가 async다	브라우저에 내려가지 않는다. async, defer, type="module"로 미루는 스크립트가 아니다. 문서 파싱과 무관하게 응답이 만들어지기 전에 끝난다
lgcns-fe-next/config/initial.ts	const로 목록과 함수를 만들고, 파일 끝에서 export { authUrlList, safeCallbackPath, withoutAuthUrlList }	프록시가 import하는 순간 서버에서 평가된다. 같은 모듈을 클라이언트 LoginSection도 import하므로 브라우저 번들에도 들어간다	default export가 없다. import x from "./config/initial"은 실패한다. 브라우저에서는 Next가 만든 모듈 그래프의 일부로 평가된다
lgcns-fe-next/app/(auth)/login/page.tsx	export default function Login() 서버 컴포넌트. 안에 <Suspense>	프록시가 NextResponse.next()를 돌려 요청을 통과시킨 뒤에만 렌더된다. 리다이렉트면 이 파일은 실행되지 않는다	페이지 HTML을 서버가 만든다. 클릭 핸들러가 없다
lgcns-fe-next/component/templates/LoginSection.tsx	첫 줄 "use client". export default function LoginSection()	로그인 페이지 HTML이 온 뒤 브라우저가 hydrate할 때 useSearchParams()가 돈다. signIn은 버튼 클릭 또는 폼 제출 때 돈다	"use client"는 <script async>도 <script defer>도 아니다. Next 번들의 클라이언트 모듈이다. useSearchParams 때문에 부모에 <Suspense>가 있다
lgcns-fe-next/lib/authOptions.ts	export const authOptions	/api/auth/... 라우트와 세션 콜백이 import할 때 서버에서 평가된다. proxy.ts는 이 파일을 import하지 않는다	프록시 요청 경로와 로그인 API 경로는 서로 다른 진입점이다
lgcns-fe-next/lib/readAccessToken.ts	export async function readAccessToken	이 함수를 부르는 서버 렌더에서 await connection() 다음에 getToken을 호출한다. 현재 proxy.ts와 로그인 페이지는 이 함수를 부르지 않는다	렌더 중 쿠키 읽기라서 connection()이 필요하다. 프록시는 이미 요청 객체 위에서 동작하므로 그 호출이 없다

HTML에서 `async`는 스크립트를 받는 즉시 실행하고 파싱을 막지 않으며 순서를 보장하지 않는다. `defer`는 문서를 파싱한 뒤 `DOMContentLoaded` 전에 적힌 순서대로 실행한다. `type="module"`은 기본적으로 지연 실행되는 ES 모듈이고, `import/export`를 쓴다. 이 앱의 접근 제어는 그 속성으로 붙지 않는다. 요청이 앱 라우트에 닿기 전에 `proxy` 함수가 끝나야 페이지가 그려진다.

참고: HTML script 요소 (https://html.spec.whatwg.org/multipage/scripting.html#the-script-element), MDN script (https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/script), Next.js proxy 파일 규약 (https://nextjs.org/docs/app/api-reference/file-conventions/proxy). 로컬 사본은 `lgcns-fe-next/node_modules/next/dist/docs/01-app/03-api-reference/03-file-conventions/proxy.md`다.

2. 이름과 런타임

Next 문서가 `middleware`를 `proxy`로 바꾼 이유는 두 가지다. Express 미들웨어와 헷갈리고, 이 기능은 앱 앞의 네트워크 경계에서 리다이렉트·리라이트·헤더를 다루는 일에 가깝기 때문이다. 이전 파일에서 옮길 때는 `npx @next/codemod@canary middleware-to-proxy .` 가 파일 이름과 함수 이름을 같이 바꾼다.

이 버전의 프록시는 Node.js 런타임이 기본이다. 라우트 세그먼트에 쓰는 `export const runtime`은 프록시 파일에서 쓸 수 없고, 넣으면 오류가 난다.

실행 순서는 `next.config`의 `headers`, `redirects` 다음이고, 파일 시스템 라우트(`app/`, `public/`, `_next/static/`)보다 앞이다. 그래서 프록시가 `redirect`를 반환하면 해당 페이지 컴포넌트는 렌더되지 않는다.

공식 문서가 같이 적는 한계: 서버 함수(Server Function)는 별도 URL이 아니라 그 함수를 쓰는 페이지로의 POST다. 매처가 그 경로를 빼면 그 페이지의 서버 함수도 프록시를 타지 않는다. 데이터 변경의 권한 검사는 프록시에만 기대지 말고 서버 함수 안에서도 해야 한다. 이 저장소의 `proxy.ts`는 헤더를 심거나 쿠키를 구우지 않는다. 통과는 `NextResponse.next()`, 이동은 `NextResponse.redirect()`만 반환한다.

참고: Proxy (https://nextjs.org/docs/app/api-reference/file-conventions/proxy), NextResponse (https://nextjs.org/docs/app/api-reference/functions/next-response).

3. 매처

```
export const config = {
    matcher: ["/((?!api|_next/static|_next/image|favicon.ico).*)"],
};
```

`matcher`가 없으면 프록시는 정적 파일과 이미지 최적화까지 포함한 모든 요청에서 돈다. 위 식은 부정 탐색(`(?!...)`)이다. 경로가 `api`, `_next/static`, `_next/image`, `favicon.ico`로 시작하면 프록시를 호출하지 않는다. 그래서 `/api/auth/...`의 NextAuth 라우트는 이 파일의 로그인 검사를 받지 않는다. CSS, JS, 이미지 요청도 여기서 막히지 않는다.

매처 값은 빌드 때 정적으로 분석된다. 변수로 넣으면 무시된다. 패턴은 `/`로 시작해야 하고, 이 앱은 문자열 하나짜리 배열을 쓴다. `has` / `missing` / `locale` 객체 형태는 쓰지 않는다.

참고: path-to-regexp (https://github.com/pillarjs/path-to-regexp#path-to-regexp-1). 문서의 negative matching 예와 같은 계열이다. 이 앱의 식에는 `sitemap.xml`, `robots.txt`, `.*\.png$`는 없다. 그 파일 요청은 매처에 걸리면 프록시 함수까지 들어온다. 함수는 그 경로를 목록과 비교한 뒤 `NextResponse.next()`로 통과시킨다.

4. 로그인으로 보는 값

```
const token = await getToken({ req: request });
const accessToken = token?.accessToken;
const loggedIn = typeof accessToken === "string" && accessToken.length > 0;
```

`getToken`은 `next-auth/jwt`의 named export다. `req`가 없으면 `Must pass req to JWT getToken()`을 던진다. 이 호출은 `secret`과 `cookieName`을 넘기지 않는다. 패키지 기본값은 다음과 같다 (`node_modules/next-auth/jwt/index.js`).

- `secret`: `process.env.NEXTAUTH_SECRET`, 없으면 `process.env.AUTH_SECRET`
- 쿠키 이름: `NEXTAUTH_URL`이 `https://`로 시작하거나 `VERCEL`이 있으면 `__Secure-next-auth.session-token`, 아니면 `next-auth.session-token`
- 쿠키가 없으면 `Authorization: Bearer ...`도 본다. 토큰 문자열이 없으면 `null`. 디코드에 실패해도 `null`

따라서 "세션 쿠키가 있다"와 "`loggedIn`이 true"는 다르다. JWT를 풀어서 `accessToken`이 비어 있지 않은 문자열일 때만 로그인으로 친다. `getServerSession`은 프록시에서 부르지 않는다. 프록시는 렌더 중의 세션 콜백을 돌리지 않고, 요청 쿠키의 JWT payload만 읽는다.

그 payload를 채우는 곳은 `lgcns-fe-next/lib/authOptions.ts`의 `jwt` 콜백이다. 프록시가 그 파일을 import하지는 않지만, 다음에 들어오는 쿠키의 내용은 그 콜백이 결정한다.

- 카카오: `account.provider === "kakao"`이면 `token.accessToken = account.access_token`을 넣고 `return token` 한다. 이 분기는 다음 `if (user)`로 떨어지지 않는다.
- 자격 증명: `authorize`가 `accessToken`을 담은 user를 반환하면, 아래 `if (user)`가 `token.accessToken = user.accessToken`으로 복사한다.
- 구글: `account.provider === "google"` 분기에서 `token.accessToken = account.access_token`을 넣은 뒤 `return`하지 않는다. 이어서 `if (user)`가 `token.accessToken = user.accessToken`으로 덮어쓴다. 구글 `user`에는 앱이 `accessToken`을 넣지 않으므로, 이 필드는 비게 된다. 그때 프록시의 `loggedIn`은 false다. 구글 로그인 직후에도 `/login` 차단과 `/cart` 허용이 기대와 다를 수 있다. 이것은 프록시 조건이 아니라 `jwt` 콜백의 현재 분기 때문이다.

`session` 콜백의 `console.log`는 프록시 경로에서 찍히지 않는다. 그 콜백은 `getServerSession`이나 NextAuth 세션 조회가 돌 때 실행된다.

참고: NextAuth getToken (https://next-auth.js.org/tutorials/securing-pages-and-api-routes#using-gettoken), JWT callback (https://next-auth.js.org/configuration/callbacks#jwt-callback).

5. 목록과 세 갈래

lgcns-fe-next/config/initial.ts:

```
const withoutAuthUrlList = ["/login", "/signup"];
const authUrlList = ["/cart", "/order"];
```

includes는 경로 전체와 같은지 본다. /login/extra나 /cart/1은 두 목록 어디에도 없다.

safeCallbackPath는 다음이면 "/"를 반환한다.

- 값이 없거나, /로 시작하지 않거나, //로 시작할 때. https://다른사이트와 //다른사이트는 여기서 떨어진다.
- ? 앞의 pathname이 withoutAuthUrlList에 있을 때. 로그인 페이지로 다시 보내는 루프를 막는다.

그 외에는 쿼리를 포함한 원래 문자열을 그대로 반환한다.

프록시 본문 순서는 고정이다.

1. pathname이 withoutAuthUrlList에 있고 로그인 전이면 NextResponse.next(). 게스트는 /login, /signup을 연다.
2. 그 경로이고 로그인이면 callbackUrl 쿼리를 safeCallbackPath에 넣은 뒤 NextResponse.redirect(new URL(callbackUrl, request.url)). 쿼리가 없거나 위험하면 /다.
3. pathname이 authUrlList에 있고 로그인 전이면 /login을 만들고 callbackUrl에 ${pathname}${request.nextUrl.search}를 넣는다. /cart?item=1이면 값은 /cart?item=1이다. search는 ?를 포함한다.
4. 위 조건이 아니면 NextResponse.next(). /, /feed, /product/..., /blog/..., /client-login은 현재 어느 목록에도 없어서 로그인 여부와 상관없이 통과한다.

NextResponse.redirect는 상태 코드를 생략하면 307이다 (node_modules/next/dist/server/web/spec-extension/response.js의 init == null ? 307). 307은 일시 리다이렉트이고, 메서드와 본문을 유지한다. 이 호출은 두 번째 인자를 넘기지 않으므로 307이다. Location은 new URL(상대경로, request.url)로 요청과 같은 출처의 절대 URL이 된다.

app/ 아래에는 /cart, /order의 page.tsx가 없다. 로그인된 요청은 프록시를 통과한 뒤 Next가 404를 낸다. 프록시가 404를 만드는 것은 아니다. 게스트의 /cart, /order는 페이지가 없어도 3번에서 로그인으로 보낸다.

/client-login 페이지는 있다. 목록에 없으므로 게스트 전용 차단도, 회원 전용 강제도 하지 않는다.

6. 로그인 이후의 주소는 프록시가 아니다

로그인 페이지(app/(auth)/login/page.tsx)는 세션을 읽지 않고 redirect()도 호출하지 않는다. <Suspense> 안에 LoginSection만 둔다.

LoginSection은 클라이언트에서 이렇게 읽는다.

```
const callbackUrl = safeCallbackPath(useSearchParams().get("callbackUrl"));
```

그 문자열을 LoginForm, KakaoLoginButton, GoogleLoginButton에 넘긴다. 셋 다 next-auth/react의 signIn을 { redirect: true, callbackUrl }와 함께 호출한다. 공급자 id는 각각 "credentials", "kakao", "google"이다. 카카오 버튼은 현재 주석이 아니고, 로그인 섹션이 렌더한다.

로그인에 성공한 뒤의 이동은 NextAuth callbacks.redirect다 (authOptions.ts).

```
async redirect({ url, baseUrl }) {
  if (url.startsWith("/")) return `${baseUrl}${url}`;
  if (new URL(url).origin === baseUrl) return url;
  return baseUrl;
}
```

상대 경로는 baseUrl에 붙이고, 같은 출처의 절대 URL은 허용하고, 다른 출처는 baseUrl만 반환한다. pages.signIn은 "/login"이라 NextAuth의 기본 로그인 화면도 이 경로다.

시점	담당	현재 코드
이미 로그인된 사람이 /login 또는 /signup을 연다	proxy.ts	safeCallbackPath 결과로 307
로그인 안 된 사람이 /cart 또는 /order를 연다	proxy.ts	/login?callbackUrl=원래경로로 307
폼 또는 OAuth 버튼으로 로그인에 성공한다	signIn(..., { callbackUrl })와 callbacks.redirect	같은 출처의 callbackUrl, 아니면 사이트 루트
목록에 없는 경로	프록시가 통과	페이지가 있으면 그 페이지, /cart·/order는 회원만 통과 후 404

프록시는 rewrite, 요청 헤더 수정, 응답 쿠키, NextFetchEvent.waitUntil을 쓰지 않는다. 두 번째 인자 event도 받지 않는다. waitUntil은 응답을 보낸 뒤에도 Promise가 끝날 때까지 프록시 수명을 늘리는 API이고, FetchEvent (https://developer.mozilla.org/docs/Web/API/FetchEvent)를 Next가 확장한 것이다. 이 파일에는 그 호출이 없다.

readAccessToken도 getToken을 쓰지만, 렌더용으로 cookies()를 평범한 객체로 감싼 뒤 넘긴다. 로그인 게이트를 페이지에서 프록시로 옮긴 뒤 로그인 페이지는 이 함수를 부르지 않는다.

7. 요청 예시

아래는 현재 분기만으로 정해지는 결과다. 구글 로그인은 4절의 accessToken 덮어쓰기 때문에 loggedIn이 false일 수 있다. 그 경우는 "로그인된 사람" 줄이 아니라 "로그인 전" 줄을 탄다.

요청	세션의 accessToken	결과
`GET /login`	없음	200, 로그인 페이지
`GET /signup`	없음	200, 가입 페이지
`GET /login`	비어 있지 않은 문자열	307, `/`
`GET /login?callbackUrl=/cart`	비어 있지 않은 문자열	307, `/cart` (페이지 파일이 없으면 그 다음 404)
`GET /login?callbackUrl=https://evil.example`	비어 있지 않은 문자열	307, `/`
`GET /login?callbackUrl=//evil.example`	비어 있지 않은 문자열	307, `/`
`GET /login?callbackUrl=/signup`	비어 있지 않은 문자열	307, `/`
`GET /cart`	없음	307, `/login?callbackUrl=%2Fcart`
`GET /cart?item=1`	없음	307, `/login?callbackUrl=%2Fcart%3Fitem%3D1`
`GET /order`	없음	307, `/login?callbackUrl=%2Forder`
`GET /` 또는 `GET /feed`	있거나 없음	프록시는 통과

`/api/auth/signin/google` 같은 경로는 매처가 `api`를 빼므로 이 함수에 들어오지 않는다.

참고: MDN 307 Temporary Redirect (https://developer.mozilla.org/docs/Web/HTTP/Reference/Status/307), NextAuth redirect callback (https://next-auth.js.org/configuration/callbacks#redirect-callback), NextAuth signIn (https://next-auth.js.org/getting-started/client#signin), URL 생성자 (https://developer.mozilla.org/docs/Web/API/URL/URL).